Year 11 Programming – Maintainable & Robust Programs

⏱️ Section 1 (15 minutes)

Maintainability: Naming, Indentation & Commenting

Why this matters

A program that works but is hard to read is difficult to fix, improve, or share with others.

Good programmers: - Use **clear variable names** - Indent code correctly - Add **useful comments** to explain what the code does

---

Challenge 1: Improving Code Readability

The Python program below works, but it is **hard to read**.

```
n=input("Enter name")
p=input("Enter password")
if n=="admin" and p=="1234":print("Welcome")
else:print("Access denied")
```

Tasks:

1. Rewrite the program so that:

    - Variable names are meaningful

    - Indentation is correct

    - Comments are added to explain each section

2. The program should still work in the same way.

⏱️ Section 2 (15 minutes)

Maintainability: Subprograms (Functions)

Why this matters

Subprograms:

- Avoid repeating code
- Make programs easier to test
- Make programs easier to change later

Challenge 2: Using a Subprogram

This program asks for two numbers and prints the total.

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print(num1 + num2)
```

Tasks:

1. Create a **function** called `calculate_total` that:
   - Takes two numbers as parameters
   - Returns the total
2. Call the function from the main program
3. Add a comment to explain what the function does

⏱️ Section 3 (15 minutes)

Robust Programs: Input Validation

Why this matters

Users do not always enter data correctly. A robust program:

- Checks input
- Rejects invalid data
- Does not crash

Challenge 3: Handling Invalid Data

Write a program that:

- Asks the user to enter their **age**
- Only accepts values **greater than 0**

- Keeps asking until a valid age is entered
- Outputs the valid age

Hints:

- Use a `while` loop
- Use comments to explain your logic

⏱️ Section 4 (15 minutes)

Robust Programs: Authentication

Why this matters

Authentication is used to **confirm the identity of a user** before allowing access to a system.

Challenge 4: Simple Authentication (Username & Password)

Write a program that:

- Stores a **correct username and password**
- Asks the user to enter a username and password t
- Allows access **only if both are correct**
- Repeats until the correct details are entered
- Uses comments to explain each part of the program

Extension: - Count how many attempts the user makes

✅ Success Criteria (Student Checklist)

Before submitting your work, check:

☐ I used clear variable names

☐ My indentation is correct

☐ I added useful comments (not obvious ones)

☐ My program handles invalid input

☐ My authentication system works correctly

Challenge yourself: > Could someone else understand and improve your program without asking you questions?

If yes — your program is maintainable 👍